

Section B — Filtering & Optimization Documentation

This section focuses on filtering records using SQL conditions and understanding optimization concepts using indexes.

The main objective of this section is to practice:

- Filtering data using the WHERE clause
- Applying multiple conditions using AND
- Filtering records using date ranges
- Understanding how indexes improve query performance
- Writing optimized and index-friendly SQL queries

This section helps in improving both query accuracy and performance.

Database Selection

Query

```
USE shopease_db;
```

Explanation

This command selects the ShopEase database before executing any queries.

It ensures that all queries run on the correct database.

Q7. Retrieve all orders with status = 'Delivered'

Query

```
SELECT *  
FROM orders  
WHERE status = 'Delivered';
```

Explanation

This query retrieves all records from the orders table where the order status is 'Delivered'.

The WHERE clause filters only those rows that match the condition.

Here:

- SELECT * fetches all columns
- WHERE status = 'Delivered' filters only delivered orders

Purpose

Helps identify successfully completed customer orders.

Expected Output

Displays all delivered orders with:

- order_id
- customer_id
- order_date
- status
- total_amount

Screenshot

The screenshot shows a SQL IDE interface with a query editor and a result grid. The query editor contains the following SQL code:

```

8
9  -- =====
10 -- Q7. Retrieve all orders with status = 'Delivered'
11 -- =====
12 • SELECT *
13   FROM orders
14  WHERE status = 'Delivered';
15

```

The result grid displays the following data:

	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1008	103	2024-08-20	Delivered	899.00
	1010	108	2024-08-28	Delivered	1598.00
•	1005	1005	1005	1005	1005

Q8. Find all Electronics products with price greater than ₹2000

Query

SELECT *

FROM products

WHERE category = 'Electronics'

AND unit_price > 2000;

Explanation

This query filters products based on two conditions:

1. Category must be Electronics
2. Unit price must be greater than 2000

The AND operator ensures both conditions must be true.

Purpose

Helps identify higher-priced electronic products.

Useful for premium product analysis.

Expected Output

Displays electronics products above ₹2000.

Screenshot

The screenshot shows a database query editor interface. The query editor window displays the following SQL query:

```
-- =====
-- Q8. Find all Electronics products with price > 2000
-- =====
SELECT *
FROM products
WHERE category = 'Electronics'
AND unit_price > 2000;
```

Below the query editor, the 'Result Grid' is visible, showing the results of the query. The grid has the following columns: product_id, product_name, category, brand, unit_price, and stock_qty. The results are as follows:

product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999.00	150
205	Bluetooth Speaker	Electronics	JBL	3499.00	200
NULL	NULL	NULL	NULL	NULL	NULL

The interface also includes a toolbar with various icons for file operations, a 'Limit to 1000 rows' dropdown, and a 'Filter Rows' input field. On the right side, there are buttons for 'Result Grid', 'Form Editor', and 'Field Types'.

Q9. Customers joined in 2024 and belong to Maharashtra

Query

```
SELECT *  
  
FROM customers  
  
WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31'  
  
AND state = 'Maharashtra';
```

Explanation

This query filters customers based on:

1. Join date in the year 2024
2. State equal to Maharashtra

BETWEEN is used for date range filtering.

The query returns only customers satisfying both conditions.

Purpose

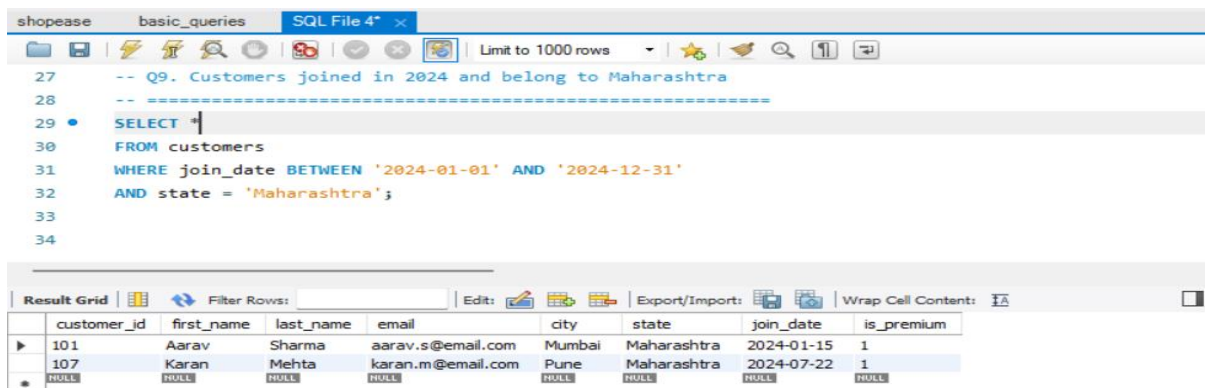
Useful for regional customer analysis.

Helps identify customer growth in Maharashtra.

Expected Output

Displays customer details of Maharashtra customers who joined in 2024.

Screenshot



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```
-- Q9. Customers joined in 2024 and belong to Maharashtra  
--  
SELECT *  
FROM customers  
WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31'  
AND state = 'Maharashtra';
```

The result grid displays the following data:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1

Q10. Orders placed between 2024-08-10 and 2024-08-25 excluding cancelled orders

Query

```
SELECT *  
  
FROM orders  
  
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'  
  
AND status != 'Cancelled';
```

Explanation

This query filters orders within a specific date range while excluding cancelled orders.

Conditions:

- Order date between the given dates
- Status is not Cancelled

!= means not equal.

Purpose

Helps analyze active order activity during a selected time period.

Expected Output

Shows only valid orders within that date range.

Screenshot

The screenshot shows a SQL IDE interface with a query editor and a results grid. The query editor contains the following SQL code:

```
-- Q10. Orders placed between 2024-08-10 and 2024-08-25  
-- Excluding cancelled orders  
SELECT *  
FROM orders  
WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25'  
AND status != 'Cancelled';
```

The results grid displays the following data:

order_id	customer_id	order_date	status	total_amount
1004	101	2024-08-10	Delivered	3499.00
1006	105	2024-08-15	Delivered	5898.00
1007	106	2024-08-18	Pending	1299.00
1008	103	2024-08-20	Delivered	899.00
1009	107	2024-08-25	Shipped	6098.00

The interface also includes a toolbar with various icons for file operations, a 'Limit to 1000 rows' dropdown, and a 'Result Grid' button on the right side.

Q11. Understanding Index: idx_orders_date

Existing Index

```
CREATE INDEX idx_orders_date ON orders(order_date);
```

Explanation

This index is created on the order_date column.

Indexes improve query speed by reducing the number of rows scanned.

Without an index:

MySQL performs a Full Table Scan.

With an index:

MySQL directly finds matching records faster.

This is especially useful for large tables.

Query using this index

```
SELECT *  
FROM orders  
WHERE order_date = '2024-08-15';
```

Explanation

This query searches for orders on a specific date.

Since order_date is indexed, MySQL can quickly locate matching rows.

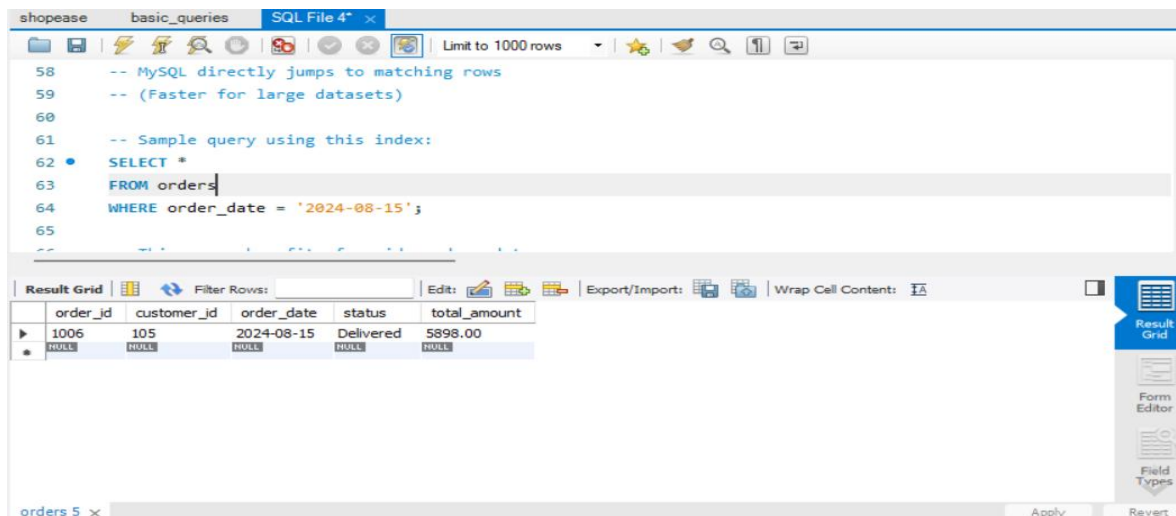
Purpose

Improves performance for date-based filtering.

Expected Output

Orders placed on 15th August 2024.

Screenshot



Q12. Index Usage on YEAR(join_date)

Query (Not Optimized)

```
SELECT *
FROM customers
WHERE YEAR(join_date) = 2024;
```

Explanation

This query extracts the year from join_date using the YEAR() function.

Problem:

Applying a function directly on a column prevents MySQL from using an index efficiently.

This causes a Full Table Scan.

This is called a **non-SARGable query**.

SARGable means:

Search Argument Able.

It means the query can use indexes efficiently.

Optimized Query (SARGable)

Query

```
SELECT *  
  
FROM customers  
  
WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31';
```

Explanation

This query directly compares the original column values without applying a function.

This makes the query index-friendly.

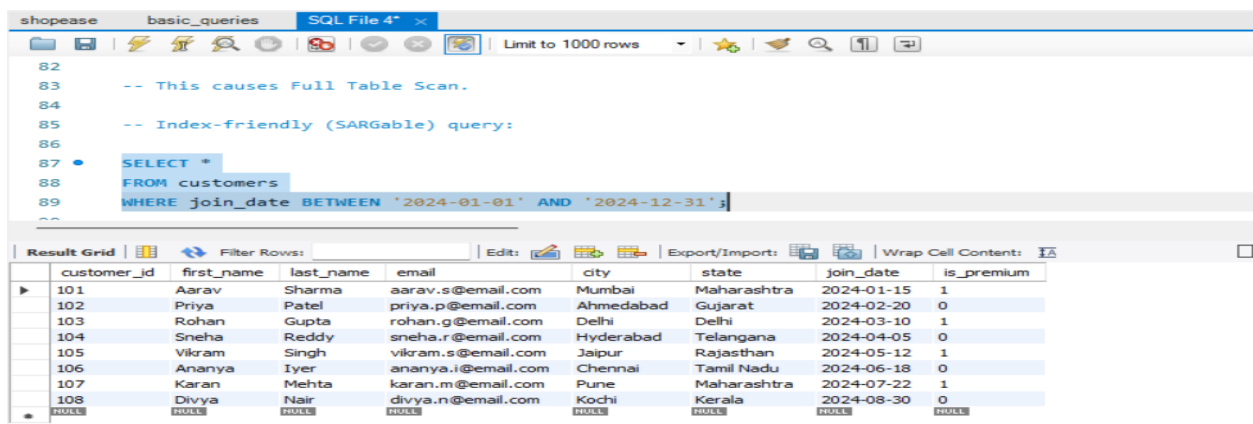
Benefits:

- Faster execution
- Better performance
- Uses index if available

Purpose

Improves filtering efficiency on date columns.

Screenshot



The screenshot shows a SQL IDE window with a query editor and a result grid. The query editor contains the following SQL code:

```
-- This causes Full Table Scan.  
  
-- Index-friendly (SARGable) query:  
  
SELECT *  
FROM customers  
WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31';
```

The result grid displays the following data:

customer_id	first_name	last_name	email	city	state	join_date	is_premium
101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0

Key Concepts Covered

- WHERE clause
- AND operator
- Date filtering using BETWEEN
- Not equal operator (!=)
- Indexing

- Query optimization
 - Full Table Scan
 - SARGable queries
-

Summary

In this section, filtering techniques were used to retrieve specific records based on conditions.

The section also introduced indexing and query optimization concepts, showing how properly written queries can improve performance.

These concepts are important for handling large datasets efficiently in real-world database systems.